

ANDROID STUDIO

KOTLIN-LEHENENGO PROIEKTUA

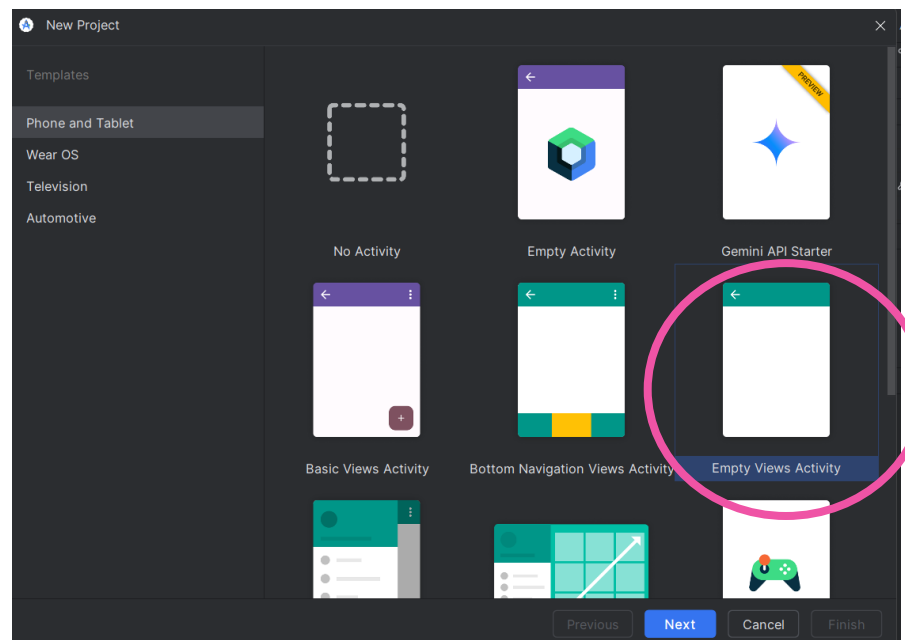
HELBURUA

Praktika honen helburu nagusia:

1. Aplikazio bat nola sortzen den ikustea
2. Oinarrizko kontzeptuak ulertzea

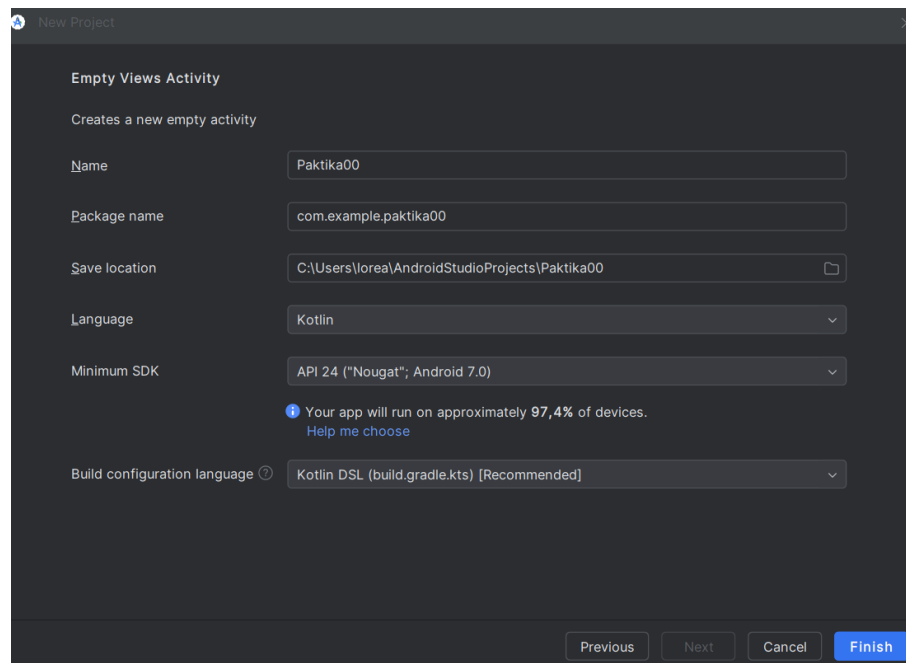
PRAKTIKA

Lehenik eta behin proiektu berri bat sortuko dugu eta horretarako gomendatzen den aukera da **Empty View Activity**.



PRAKTIKA

Proiektu honeri **Praktika00** deituko diogu eta gure gainontzeko proiektuak gorde ditugun toki berdinean gordeko dugu. Behin Finish-eri emonda proiektua sortzen hasiko da, pazientzia mesedez pixkat geldoa baita.



The screenshot shows the 'New Project' dialog in Android Studio. The dialog is titled 'New Project' and has a close button (X) in the top right corner. It contains the following fields and options:

- Empty Views Activity**: A section header.
- Creates a new empty activity**: A description of the project type.
- Name**: A text field containing 'Paktika00'.
- Package name**: A text field containing 'com.example.paktika00'.
- Save location**: A text field containing 'C:\Users\lorea\AndroidStudioProjects\Paktika00' with a folder icon to its right.
- Language**: A dropdown menu with 'Kotlin' selected.
- Minimum SDK**: A dropdown menu with 'API 24 ("Nougat"; Android 7.0)' selected.
- Build configuration language**: A dropdown menu with 'Kotlin DSL (build.gradle.kts) [Recommended]' selected.

Below the 'Minimum SDK' field, there is a blue information icon and the text: 'Your app will run on approximately 97,4% of devices. Help me choose'.

At the bottom of the dialog, there are four buttons: 'Previous', 'Next', 'Cancel', and 'Finish'.

PRAKTIKA

Ondoren egiaztatuko dogu ondorengo fitxagegiak ditugula:

- ▶ MainActivity
- ▶ activity_main.xml (layout-an barnean)
- ▶ Android_manifest.xml

PRAKTIKA

Aplikazio bat sortzeko garaian argi izan behar dogu zein den aplikazioak izango duen itzura, hau da diseinua. Gure kasuan ondorengo diseinua egingo dugu:

Ezaugarriak:

Textview

Izena: txtTextua

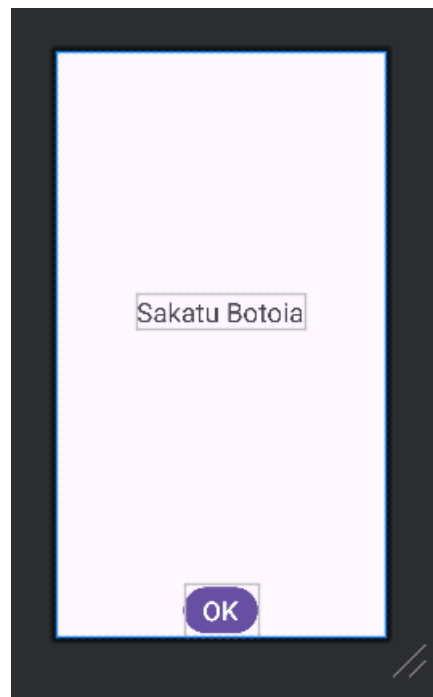
Tamainua: 34sp

Estiloa: Beltzean

Button

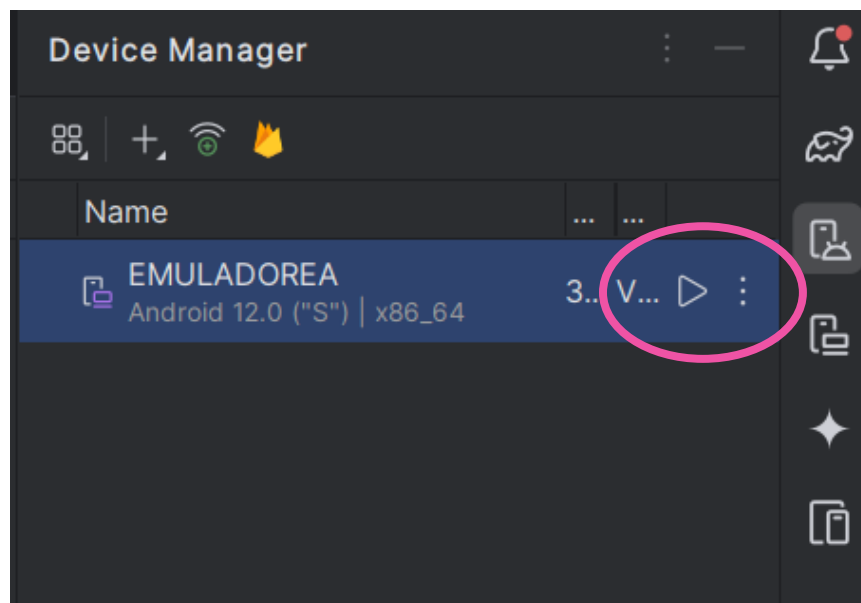
Izena: btnSakatu

Tamainua: 34sp



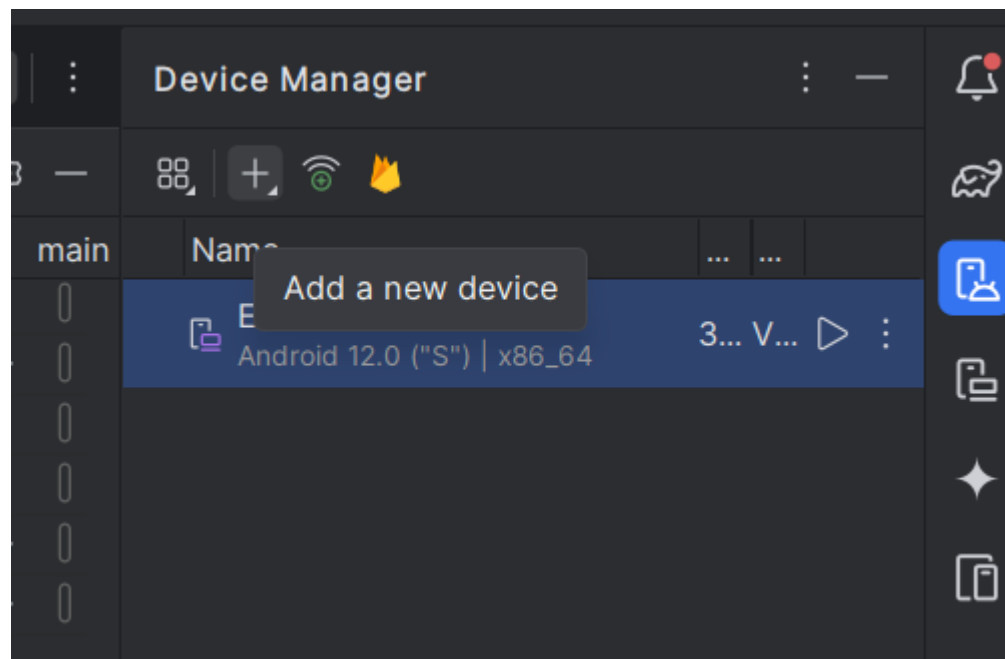
PRAKTIKA

Ezaugarrik aldatu ondoren egiaztatuko dugu gure emuladorean benetan egindakoa ondo ikusten dela. Horretarako gure emuladorea martxan jarriko dugu.

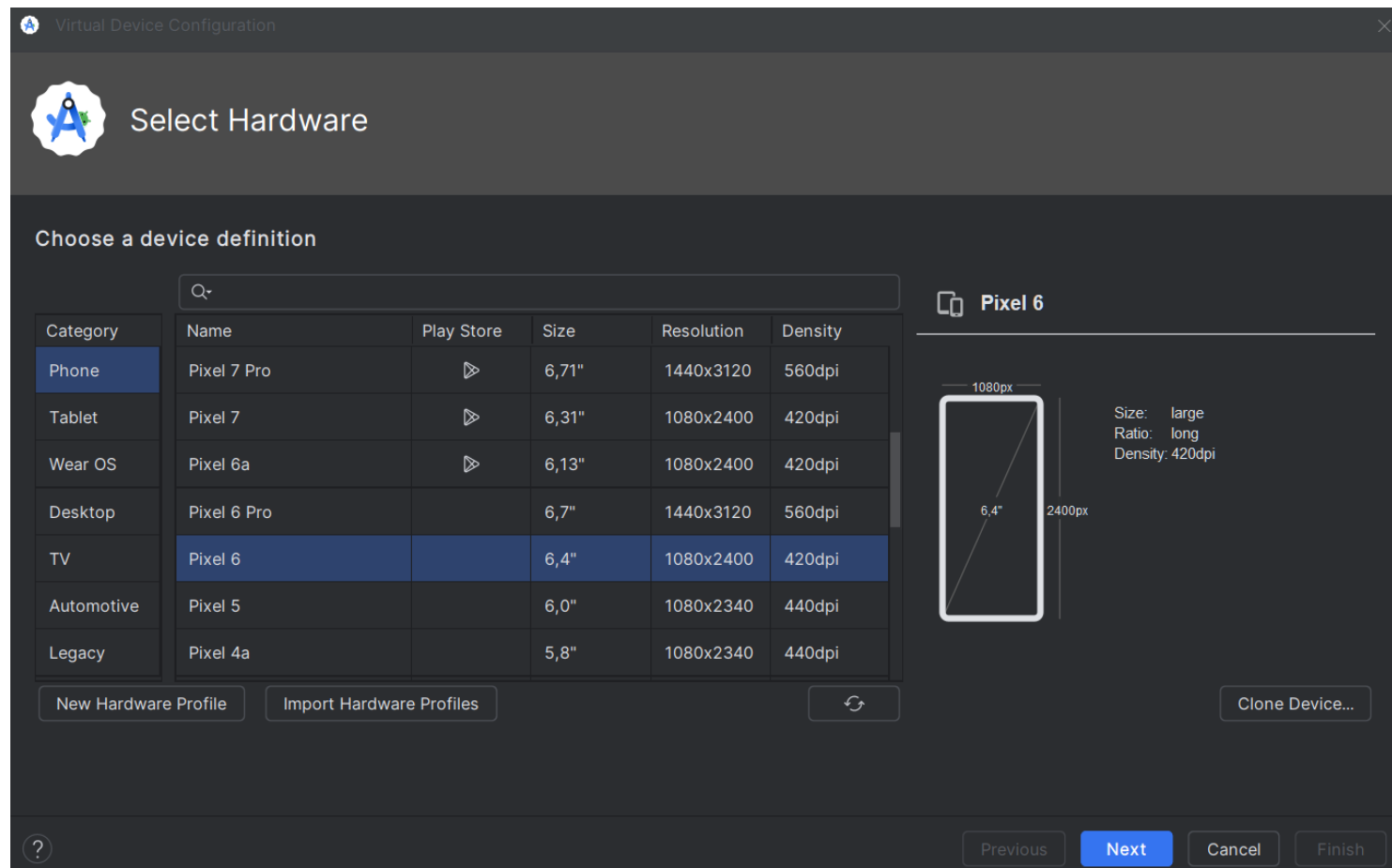


PRAKTIKA

Emuladorerik sortuta ez badago. Sortu ondorengo pausoak jarraituz.



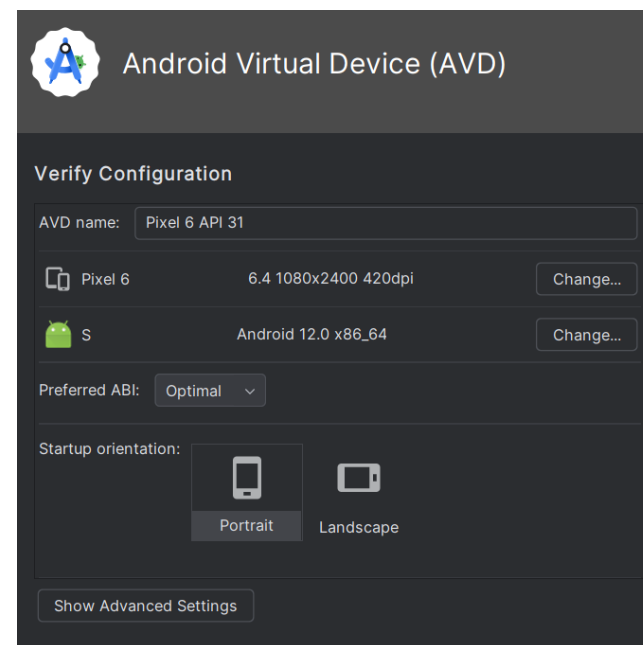
PRAKTIKA



PRAKTIKA

Ondoren sistemaren irudia aukeratu behar da, denak  deskargatzeko aukera badute S-a aukeratu adibidez eta bajatu. Behin eguneratuta badago aukeraketa besterik ez da egin behar, zure emuladoreari izena eta ezaugarriak aldatzeko emateko aukera ematen dizu.

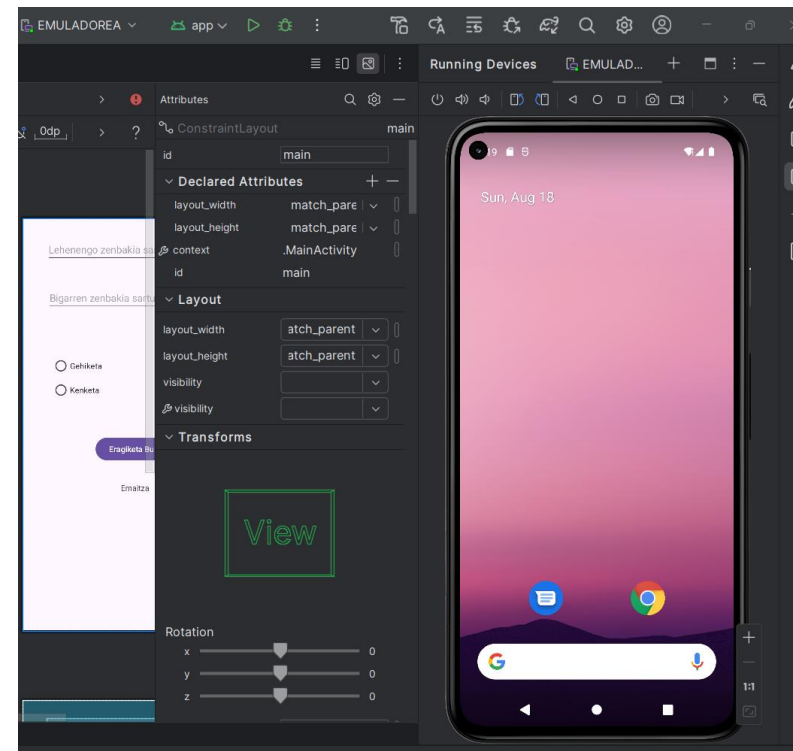
Sortzeak denbora nahikotsu suposatzen du, pazientzia izan!!!!



PRAKTIKA

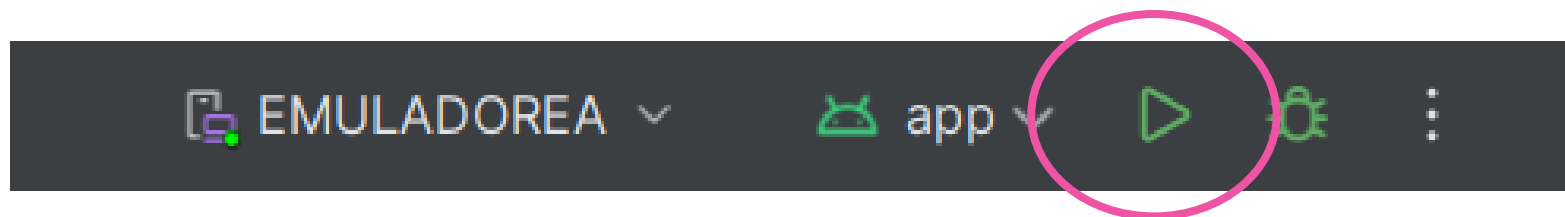
Martxan dagoenean ondorengo irudia azalduko zaigu:

```
Open logcat panel for emulator EMULADOREA  
Connected to process 7243 on device 'EMULADOREA [emulator-5554]'.
```



PRAKTIKA

Behin Emuladorea martxan dagoenean gure aplikazioa exekutatzea besterik ez zaigu falta :



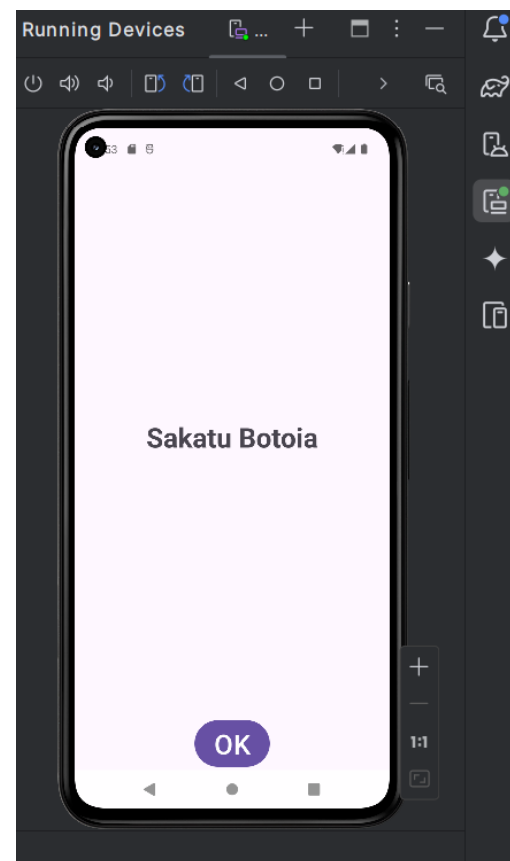
Oharra: Emuladore bat baino gehiago izanez gero nahi duguna aukeratua egon beharko du.

PRAKTIKA

Exekuzioak bere denbora
behar du. Berriz ere
PAZIENTZIA!!!

PRAKTIKA

Lortu beharreko emaitza:



PRAKTIKA

Xml-a aztertuz gero ikus genezake sortutako objetuen ezaugarriak:

- ▶ Identifikadorea: bertan gure osagaiek jarritako izenak agertzen dira. Hemendik ere aldatu daiteke. @+id/ ez ezabatu.
- ▶ Taimainua: kasu hontan testua eta botoiari egokitua dago.
- ▶ Testua: Ikusiko den testua.
- ▶ Letran tamaina eta estiloa, besterik beste.

```
<TextView
    android:id="@+id/txtTextua"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Sakatu Botoia"
    android:textSize="34sp"
    android:textStyle="bold"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.442" />
```

```
<Button
    android:id="@+id/btnSakatu"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="OK"
    android:textSize="34sp"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent" />
```

PRAKTIKA

Behin diseinua amaituta gure aplikazioaren portaera programatu beharko dugu: Xml fitxategian onclick jarri genezake baina ondorengo azaltzen da:

```
<Button
    android:id="@+id/btnSakatu"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="OK"
    on
    android:onClick
    android:layout_marginHorizontal
```

Holan agertzen den kasuetan aukerak denboragaz funtzionatzeari utziko diola adierazten du beraz komenigarria da ez erabiltzea. Beraz MainActivity.kt-ra abiatuko gara.

PRAKTIKA

Aldagai bat definituko dugu botoirako lateinit bezala jarriko dugu honek esan nahi du behin aldagai hasieratuko da definizioaren ostean.

```
lateinit var Sakatu:Button

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    //enableEdgeToEdge()
    setContentView(R.layout.activity_main)
    //ViewCompat.setOnApplyWindowInsetsListener(findVi
        //val systemBars = insets.getInsets(WindowInse
        //v.setPadding(systemBars.left, systemBars.top
        //insets

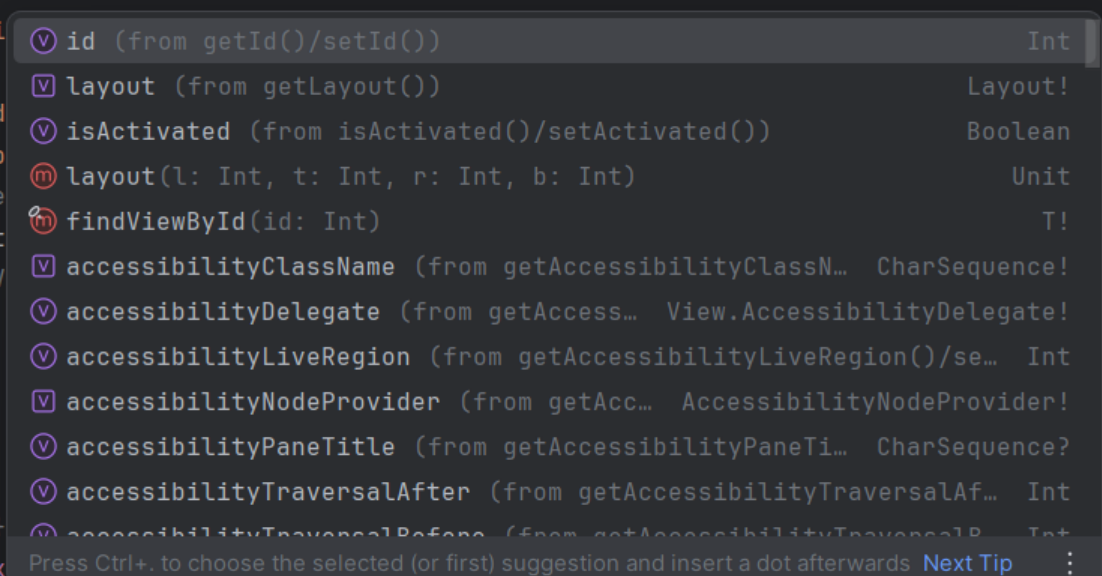
    //}
    Sakatu= findViewById(R.id.btnSakatu)
}
```

Honegaz Button osagaia Bistan barruan topatu dugu eta orain berarekin jolas genezake.

PRAKTIKA

Aldagai berriaren ostean puntu jartzean honek dituen aukera guztiak agertuko zaizkigu, funtzioak barne:

```
class MainActivity : AppCompatActivity() {  
    lateinit  
    override  
    sup  
    //e  
    set  
    //V  
    //}  
    Sak  
    Sakatu.  
}
```



Property/Method	Type
id (from getId()/setId())	Int
layout (from getLayout())	Layout!
isActivated (from isActivated()/setActivated())	Boolean
layout(l: Int, t: Int, r: Int, b: Int)	Unit
findViewById(id: Int)	T!
accessibilityClassName (from getAccessibilityClassN...	CharSequence!
accessibilityDelegate (from getAccess...	View.AccessibilityDelegate!
accessibilityLiveRegion (from getAccessibilityLiveRegion()/se...	Int
accessibilityNodeProvider (from getAcc...	AccessibilityNodeProvider!
accessibilityPaneTitle (from getAccessibilityPaneTi...	CharSequence?
accessibilityTraversalAfter (from getAccessibilityTraversalAf...	Int
accessibilityTraversalBefore (from getAccessibilityTraversalB...	Int

Press Ctrl+. to choose the selected (or first) suggestion and insert a dot afterwards Next Tip

PRAKTIKA

Bertan click idatziz gero click hitza duten aukera guztiak azalduko zaizkigu. Niri interesatzen zaidana `setOnClickListener` izengo da

```
lateini performLongClick() Boolean
(m) performLongClick(x: Float, y: Float) Boolean
overrid (v) isContextClickable (from isContextClickable()/setContextC... Boolean
sup (m) hasOnClickListeners() Boolean
//e (m) hasOnLongClickListeners() Boolean
set (m) setAllowClickWhenDisabled(clickableWhenDisabled: Boolean) Unit
//V (m) setOnClickListener(l: View.OnClickListener?) Unit
(m) setOnClickListener {...} (l: ((View!) -> Unit)?) Unit
(m) setOnContextClickListener(l: View.OnContextClickListener?) Unit
(m) setOnContextClickListener {...} (l: ((View!) -> Boolean)?) Unit
(m) setOnLongClickListener(l: View.OnLongClickListener?) Unit
//} (m) setOnLongClickListener {...} (l: ((View!) -> Boolean)?) Unit
Sak Ctrl+Abajo and Ctrl+Arriba will move caret down and up in the editor Next Tip
Sakatu.click
```

PRAKTIKA

Honek adierazten duen gauza bakarra da aplikazioa entzuten geratzen dela ia erabiltzaileak botoia sakatu duen. Sakatu duela egiaztatzeko Log erabiliko dugu.

```
Sakatu.setOnClickListener {  
    Log.d( tag: "0harra", msg: "Botoian sakatu da")  
}
```

Aplikazioa exekutatu dugu eta Botoia sakatu ondoren zer gertatzen den aztertu:

